Doramagic 产品手册 v1.0

> **最后更新**: 2026-03-11
>
> **状态**: 复盘整合版——基于全部研究文档、实验数据与代码实现的完整产品定义

目录

1. [产品概述](#1-产品概述)
2. [核心价值主张](#2-核心价值主张)
3. [目标用户](#3-目标用户)
4. [系统架构](#4-系统架构)
5. [功能清单](#5-功能清单)
6. [知识卡片体系](#6-知识卡片体系)
7. [质量保障体系](#7-质量保障体系)
8. [用户使用流程——Mac 软件版本](#8-用户使用流程mac-软件版本)
9. [用户使用流程——OpenClaw Skill 版本](#9-用户使用流程openclaw-skill-版本)
10. [输出物说明](#10-输出物说明)
11. [技术栈与依赖](#11-技术栈与依赖)
12. [验证数据与实验结论](#12-验证数据与实验结论)
13. [已知限制与待解决问题](#13-已知限制与待解决问题)
14. [产品路线图](#14-产品路线图)
15. [附录](#15-附录)

---

1. 产品概述

1.1 一句话定义

Doramagic 是一个开源项目"灵魂提取"系统——它从开源项目的代码、文档和社区经验中提取隐性知识（设计哲学、决策规则、社区踩坑），转化为结构化的知识资产，注入 AI 使其成为该项目的专家级顾问。

1.2 解决什么问题

问题	现状	Doramagic 方案
AI 对开源项目一知半解	通用 LLM 回答准确率 ~42%，经常"自信地给出错误答案"	注入灵魂后准确率提升至 96%，幻觉完全消除
99% 中小企业没有技术团队	无法阅读源码、理解架构、追踪 Issue	将项目知识转化为非技术用户可消费的 AI 顾问服务
开源知识散落各处	代码、README、Issue、Discussion、Stack Overflow、博客	统一提取、结构化、质量验证后输出
"差不多正确"的陷阱	AI 看似合理实则微妙错误的回答，非技术用户无法辨别	知识卡片带证据溯源、置信度标注、事实核查门控

1.3 核心理念——双半魂模型

Doramagic 将项目知识分为两个互补的"半魂"：

```
┌──────────────────────────────────────────────────────┐
│                    完整项目灵魂                        │
│                                                        │
│  ┌──────────────────┐  ┌──────────────────────┐      │
│  │   代码半魂        │  │   社区半魂            │      │
│  │   Code Soul       │  │   Community Soul      │      │
│  │                   │  │                       │      │
│  │ • 设计哲学        │  │ • 实践中的坑          │      │
│  │ • 心智模型        │  │ • 兼容性问题          │      │
│  │ • 架构意图        │  │ • 部署经验            │      │
│  │ • 决策规则        │  │ • 社区最佳实践        │      │
│  │ • 模块边界        │  │ • 演化故事            │      │
│  │                   │  │                       │      │
│  │ 回答: 它怎么工作的 │  │ 回答: 实际用会出什么事 │      │
│  └──────────────────┘  └──────────────────────┘      │
│                                                        │
│  单独代码半魂: 42% → 83% (+96% 提升)                   │
│  完整双半魂:   42% → 96% (+125% 提升)                  │
└──────────────────────────────────────────────────────┘
```

1.4 产品形态

Doramagic 以两种形态交付：

形态	说明	运行环境
Mac CLI 软件	独立命令行工具 `doramagic`	macOS, Node.js 18+
OpenClaw Skill	OpenClaw 平台原生技能	OpenClaw 工作空间

两种形态共享相同的提取管道和知识卡片体系，区别在于交互方式和输出集成。

---

2. 核心价值主张

2.1 对开发者

- **消除 AI 幻觉**: 将项目知识结构化注入后，AI 不再"编造"答案

- **加速项目理解**: 新人可通过 AI 顾问快速掌握项目设计哲学和决策背景
- **自动生成 CLAUDE.md / .cursorrules**: 直接加载到 Claude Code 或 Cursor IDE

2.2 对非技术用户

- **不需要读代码**: AI 顾问以自然语言解答项目相关问题
- **知道哪些坑会踩**: 社区半魂包含所有实践中发现的问题和解法
- **决策有据可依**: 每个知识点都有证据溯源（代码行号、Issue 编号、社区讨论）

2.3 竞品差异

维度	Doramagic	Copilot/Cursor	Repomix	DeepWiki	传统 RAG
社区知识提取	✅ 独有	❌	❌	部分	❌
设计哲学提取	✅	❌	❌	❌	❌
非技术用户友好	✅	❌	❌	部分	❌
知识质量验证	✅ 5维评判	❌	❌	❌	❌
证据可溯源	✅ file:line	部分	❌	❌	部分
端到端自动化	✅	N/A	仅打包	仅展示	需大量配置

---

3. 目标用户

3.1 主要用户画像

P1: 技术决策者（CTO/技术负责人）

- 需要快速评估开源项目的设计理念和技术风险
- 需要了解社区踩坑经验来做选型决策
- 用例：评估是否采用某个开源框架

P2: 开发者（个人/团队）

- 需要快速上手新项目的代码架构和设计哲学

- 需要 AI 顾问在编码时提供项目特定的精准建议
- 用例：将提取的 CLAUDE.md 加载到 IDE 中辅助开发

P3: 非技术创业者/产品经理

- 没有技术团队但需要利用开源项目的知识
- 需要理解项目能做什么、有什么限制
- 用例：通过 AI 顾问了解开源 CRM 的定制能力

3.2 使用场景

场景	用户	输入	期望输出
项目评估	CTO	GitHub URL	设计哲学报告 + 风险清单
开发辅助	开发者	GitHub URL	CLAUDE.md + .cursorrules
技术咨询	产品经理	项目名称	AI 顾问知识包
团队培训	技术lead	内部项目路径	架构说明 + 决策记录
社区调研	运维工程师	GitHub URL	踩坑清单 + 最佳实践

---

4. 系统架构

4.1 总体架构——三平面设计

```
┌──────────────────────────────────────────────────────────────┐
│                    控制平面 (Control Plane)                   │
│                                                              │
│  用户输入 → 参数解析 → 阶段调度 → 质量门控 → 输出组装        │
│  (CLI/Skill)                                                 │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                    知识平面 (Knowledge Plane)                 │
│                                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐     │
│  │ 概念卡片 │  │ 工作流卡片│  │ 决策规则 │  │ 架构卡片 │     │
│  │ CC-*     │  │ WC-*     │  │ DR-*     │  │ AC-*     │     │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘     │
│        ┌──────────┐                                          │
│        │ 合约卡片 │    ← 所有卡片: Markdown + YAML 前置      │
│        │ CT-*     │                                          │
│        └──────────┘                                          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                    执行平面 (Execution Plane)                 │
│                                                              │
│  Repomix打包 → LLM提取 → 脚本验证 → GitHub API → 输出        │
│                                                              │
│  支持: Claude / Gemini / Ollama (本地)                       │
└──────────────────────────────────────────────────────────────┘
```

4.2 提取管道——8+2 阶段流水线

```
Stage 0: 准备 (Prepare)
    │  下载仓库 → Repomix 压缩打包 → 确定性事实提取
    ▼
Stage 1: 本质发现 (Essence Discovery)
    │  7 问协议 → 设计哲学 + 心智模型
    │  输出: 00-soul.md
    ▼
Stage 2: 概念与工作流 (Concepts & Workflows)
    │  提取 3+ 概念卡片 + 3+ 工作流卡片
    │  输出: CC-*.md, WC-*.md
    ▼
Stage 3: 决策规则 (Decision Rules)
    │  从代码提取 5+ 规则 + 社区 3+ 经验
    │  输出: DR-001~*.md (代码), DR-100~*.md (社区)
    ▼
Stage 3.5: 验证门控 (Validation Gate) ★ 硬门控
    │  运行 validate_extraction.py
    │  PASS → 继续  |  BLOCKED → 修复后重试(最多2次)
    ▼
Stage 4: 专家叙事合成 (Expert Narrative)
    │  将所有卡片合成专家级叙事文档
    │  输出: expert_narrative.md (≤1500词)
    ▼
Stage M: 模块地图 (Module Map)
    │  识别 3-7 个核心模块 + 依赖关系
    │  输出: module-map.md (含 Mermaid 图)
    ▼
Stage C: 社区智慧 (Community Wisdom)
    │  GitHub Issues/Discussions 三层漏斗提取
    │  输出: community-wisdom.md
    ▼
Stage F: 组装输出 (Assembly)
    │  合并所有产出 → 生成最终交付物
    │  输出: CLAUDE.md + .cursorrules + advisor-brief.md
    ▼
  ✅ 完成
```

4.3 社区智慧提取——三层漏斗

```
Tier 1: 过滤 (Filter)
    │  GitHub Issues/PRs → 1-10 相关性评分 → 筛选高分线程
    ▼
Tier 2: 分类 (Classify)
    │  线程分类: bug-fix / workaround / best-practice / anti-pattern / migration
    ▼
Tier 3: 提取 (Extract)
    │  生成决策规则卡片 (DR-100+) + 证据级别标注 (E1-E4)
    ▼
  输出: 社区规则卡片 + 集体认知摘要
```

4.4 项目分解引擎

```
decompose 命令输出:
├── overview.md          # 项目总览
├── module-tree.md       # 模块树
├── feature-list.md      # 功能清单
├── data-flow.md         # 数据流图 (Mermaid)
├── config-reference.md  # 配置参考
└── modules/
    ├── M-001/
    │   ├── soul.md      # 模块灵魂
    │   └── api.md       # 模块 API 文档
    ├── M-002/
    │   └── ...
    └── ...
```

---

5. 功能清单

5.1 Mac CLI 软件功能

命令	功能	状态	说明
`doramagic extract `	灵魂提取	✅ 已实现	从 GitHub URL 或本地路径提取代码半魂
`doramagic decompose `	项目分解	✅ 已实现	生成模块树、功能清单、数据流图
`doramagic community `	社区智慧提取	✅ 已实现	从 GitHub Issues/PRs 提取社区经验
`doramagic full `	全流程提取	✅ 已实现	extract + decompose + community 一体化
`doramagic config`	配置查看	✅ 已实现	显示当前有效配置
`doramagic providers`	提供商管理	✅ 已实现	列出可用 LLM 提供商及状态

CLI 选项:

```
doramagic extract <source> [options]

选项:
  --provider <name>     LLM 提供商 (claude/gemini/ollama), 默认按配置文件
  --model <name>        模型名称
  --output <dir>        输出目录, 默认 ./doramagic-output/<slug>
  --language <code>     输出语言 (zh-CN/en), 默认 zh-CN
  --max-concepts <n>    最大概念卡片数, 默认 10
  --max-workflows <n>   最大工作流卡片数, 默认 5
  --evidence-level <l>  最低证据级别 (E1-E4), 默认 E2
```

5.2 OpenClaw Skill 功能

技能	触发词	功能	状态
soul-extractor	"doramagic", "extract soul", "提取灵魂", "AI 顾问"	完整 8+2 阶段灵魂提取	✅ v1.0
skill-forge	"forge skill", "锻造技能", "定制技能"	从用户需求出发发现并锻造 AI 技能	✅ v1.0

soul-extractor 触发后交互流程:

1. 用户提供 GitHub URL 或项目名称
2. 系统自动执行 Stage 0-F
3. 每个阶段输出可检视
4. 验证门控自动执行
5. 最终生成 CLAUDE.md + .cursorrules + advisor-brief.md

skill-forge 触发后交互流程:

1. 苏格拉底式对话发现用户真实需求
2. 自动搜索 3-5 个相关开源项目
3. 对每个项目执行灵魂提取
4. 合成社区智慧
5. 锻造定制 AI 技能包

5.3 完整特性清单 (42 项)

CLI 核心 (9 项):

ID	特性	状态
CLI-001	help 命令与使用说明	✅
CLI-002	version 显示	✅
CLI-003	extract 命令	✅
CLI-004	config 命令	✅
CLI-005	providers 命令	✅
CLI-006	decompose 命令	✅
CLI-007	community 命令	✅
CLI-008	full 命令	✅
CLI-009	进度 spinner 显示	✅

提取引擎 (8 项):

ID	特性	状态
EXT-001	Eagle Eye Phase 1 — 项目身份识别	✅
EXT-002	Eagle Eye Phase 2 — 模块地图发现	✅
EXT-003	Eagle Eye Phase 3 — 灵魂焦点定位	✅
EXT-004	Deep Dive L1 — 概念卡片提取	✅
EXT-005	Deep Dive L2 — 工作流卡片提取	✅
EXT-006	Deep Dive L3 — 决策规则提取	✅
EXT-007	项目分解引擎	✅
EXT-008	社区三层漏斗提取	✅

LLM 集成 (5 项):

ID	特性	状态
LLM-001	Claude (Anthropic SDK) 集成	✅
LLM-002	Gemini (Google AI SDK) 集成	✅
LLM-003	Ollama (本地模型) 集成	✅
LLM-004	LLM 错误处理与重试	✅
LLM-005	提供商动态切换	✅

知识卡片 (4 项):

ID	特性	状态
CARD-001	YAML 前置验证	✅
CARD-002	卡片 ID 唯一性校验	✅
CARD-003	证据级别标注 (E1-E4)	✅
CARD-004	中英双语输出	✅

配置管理 (3 项):

ID	特性	状态
CFG-001	doramagic.yaml 配置加载	✅
CFG-002	环境变量覆盖	✅
CFG-003	默认值回退	✅

测试 (4 项):

ID	特性	状态
TST-001	卡片模块单元测试	✅
TST-002	配置模块单元测试	✅
TST-003	LLM 模块单元测试	✅
TST-004	集成测试框架	✅

构建 (3 项):

ID	特性	状态
BLD-001	TypeScript 编译	✅
BLD-002	npm install 依赖安装	✅
BLD-003	npm link 全局安装	✅

集成测试 (2 项):

ID	特性	状态
INT-001	superpowers 端到端提取	✅
INT-002	大仓库 token 限制处理	⏳ 待验证

错误处理 (3 项):

ID	特性	状态
ERR-001	无效路径/URL 处理	✅
ERR-002	缺少 API Key 提示	✅
ERR-003	网络错误处理	✅

输出 (2 项):

ID	特性	状态
OUT-001	输出目录结构生成	✅
OUT-002	soul.md 各章节完整	✅

质量系统 (独立于特性清单):

功能	说明	状态
卡片集合验证	概念卡 5-10、工作流 ≥1、架构 ≥1	✅
Eagle Eye 基线验证	模块 >0、入口点 >0	✅
卡片引用验证	重复 ID 检测、悬空引用检测	✅
证据锚点验证	对照快照文件验证证据引用	✅
卡片形状验证	类型特定检查 (工作流步骤、合约不变量、Mermaid 图)	✅
5 维评分	Faithfulness / Coverage / Traceability / Consistency / Actionability	✅
三级判定	PASS (0错误,均分≥80) / REVISE (≤3错误,均分≥55) / FAIL	✅

---

6. 知识卡片体系

6.1 五种卡片类型

类型	ID 前缀	用途	数量要求
概念卡片 (Concept Card)	CC-	核心概念的 "是什么/不是什么" 定义	5-10
工作流卡片 (Workflow Card)	WC-	关键流程的步骤说明	≥1
决策规则卡片 (Decision Rule)	DR-	条件判断规则与踩坑经验	代码≥5, 社区≥3
合约卡片 (Contract Card)	CT-	系统不变量与约束条件	≥0
架构卡片 (Architecture Card)	AC-	系统架构全景图	≥1

6.2 卡片结构标准

每张卡片统一包含:

```
---
card_type: concept_card | workflow_card | decision_rule_card | contract_card | architectu
card_id: CC-PROJECT-NAME-001
repo: github_owner/repo_name
title: 卡片标题
---
```

概念卡片 (CC) 特有字段:

- 身份定义 (一句话)
- 是/不是 对照表
- 关键属性
- 边界说明
- 证据 (file:line 引用)

工作流卡片 (WC) 特有字段:

- 步骤列表 (含 file:line 引用)
- 流程图 (Mermaid)
- 前置条件与输出
- 失败模式

决策规则卡片 (DR) 特有字段:

- 规则声明 (IF...THEN)
- 严重等级 (critical/warning/info)
- DO / DON'T 对照
- 置信度 (0.0-1.0)

- 证据来源 (Issue #, CHANGELOG, 代码行号)
- 真实场景叙述

合约卡片 (CT) 特有字段:

- 不变量列表
- 违反后果
- 验证方式

架构卡片 (AC) 特有字段:

- 模块列表与职责
- 依赖关系 (Mermaid 图)
- 架构决策说明

6.3 证据级别体系

级别	名称	说明	可信度
E1	源代码	直接从代码中提取，有 file:line 引用	最高
E2	维护者声明	来自 maintainer 的 Issue/PR/commit 评论	高
E3	社区共识	多个用户在 Issues/Discussions 中达成的共识	中
E4	轶事证据	个别用户报告或博客文章	参考

---

7. 质量保障体系

7.1 验证门控 (Stage 3.5)

提取完成后，系统自动运行验证脚本 `validate_extraction.py`：

1. **结构验证**: 检查所有必需卡片是否存在，ID 格式是否正确
2. **引用验证**: 检查证据锚点是否指向真实存在的文件和行号
3. **一致性验证**: 检查跨卡片引用是否一致
4. **完整性验证**: 检查是否满足最低数量要求

```
判定逻辑:
  PASS    → 继续到 Stage 4
  BLOCKED → 自动修复 → 重试 (最多 2 次)
  FAIL    → 停止并报告错误
```

7.2 质量评判系统 (Judge)

最终输出通过 5 维度评分系统:

维度	权重	计算方式
忠实度 (Faithfulness)	—	100 − (错误数 × 20) − (警告数 × 4)
覆盖度 (Coverage)	—	概念卡数量 + 工作流存在 + 类型覆盖率的综合
可追溯性 (Traceability)	—	(有效锚点/总锚点 × 70) + (有源卡片/总卡片 × 30)
一致性 (Consistency)	—	100 − (错误数 × 14) − (警告数 × 6)
可操作性 (Actionability)	—	工作流(35) + 决策规则(25) + 合约(25) + 架构(15)

最终判定:

- **PASS**: 0 错误且均分 ≥ 80
- **REVISE**: ≤ 3 错误且均分 ≥ 55
- **FAIL**: 其他情况

7.3 反理性化检查表

在 Stage 3 (规则提取) 中，系统强制执行反理性化检查，防止 LLM 跳过不方便的事实:

- ✅ 是否提取了至少 5 条代码规则?
- ✅ 是否提取了至少 3 条社区踩坑经验?
- ✅ 每条社区规则是否有 Issue # 或 CHANGELOG 引用?
- ✅ 是否包含了"不直觉"的规则? （而不只是显而易见的）

7.4 确定性事实提取

Stage 0 包含确定性（非 LLM）事实提取脚本 `extract_repo_facts.py`：

- 提取 console_scripts、entry_points
- 提取模块结构和文件统计
- 提取依赖信息
- 输出 `repo_facts.json` 作为后续 LLM 提取的校准基准

---

8. 用户使用流程——Mac 软件版本

8.1 安装

```
# 前置要求
node --version  # >= 18.0.0
npm --version   # >= 8.0.0

# 安装
cd doramagic-codex
npm install
npm run build
npm link       # 全局安装 doramagic 命令

# 验证
doramagic --version
doramagic providers  # 查看 LLM 提供商状态
```

8.2 配置

创建或编辑 `doramagic.yaml`：

```
llm:
  provider: claude          # claude / gemini / ollama
  model: claude-sonnet-4-20250514
  # base_url: http://localhost:11434  # Ollama 专用

extraction:
  max_concepts: 10          # 最大概念卡片数
  max_workflows: 5          # 最大工作流卡片数
  evidence_level: E2        # 最低证据级别

community:
  min_comments: 5           # Issue 最少评论数
  require_maintainer: true  # 是否要求维护者参与
  max_threads: 100          # 最大分析线程数

output:
  format: markdown          # 输出格式
  language: zh-CN           # 输出语言
```

环境变量（按需设置）：

```
export ANTHROPIC_API_KEY="sk-ant-..."       # Claude
export GOOGLE_AI_API_KEY="AI..."            # Gemini
# Ollama 无需 API Key（本地运行）
```

8.3 基本使用流程

场景 1: 完整灵魂提取（推荐）

```
# 对 GitHub 项目执行完整提取
doramagic full https://github.com/theskumar/python-dotenv

# 指定 LLM 提供商
doramagic full https://github.com/theskumar/python-dotenv --provider claude

# 指定输出目录
doramagic full https://github.com/theskumar/python-dotenv --output ./my-output
```

执行过程中会看到进度指示：

```
⠋ Preparing repository...
✔ Repository prepared (1,105 lines, 23 files)
⠋ Running Eagle Eye analysis...
✔ Identity discovered: python-dotenv
✔ Module map: 4 modules identified
✔ Soul locus: 3 zones identified
⠋ Running Deep Dive extraction...
✔ Concept cards: 6 extracted
✔ Workflow cards: 3 extracted
✔ Decision rules: 8 extracted
⠋ Running quality validation...
✔ Quality gate: PASS (avg: 96)
⠋ Extracting community wisdom...
✔ Community rules: 21 extracted
⠋ Assembling final output...
✔ Output saved to ./doramagic-output/python-dotenv/

Generated files:
  soul.md              Project soul summary
  cards/               14 knowledge cards
  modules/             4 module analyses
  judge-report.md      Quality report (PASS)
  CLAUDE.md            For Claude Code
  .cursorrules         For Cursor IDE
  advisor-brief.md     For non-technical users
```

场景 2: 仅提取代码半魂

```
doramagic extract https://github.com/theskumar/python-dotenv
```

场景 3: 仅分析项目结构

```
doramagic decompose https://github.com/theskumar/python-dotenv
```

场景 4: 仅提取社区经验

```
doramagic community https://github.com/theskumar/python-dotenv
```

场景 5: 本地项目提取

```
doramagic full /path/to/local/project
```

8.4 使用提取结果

加载到 Claude Code:

```
# 将 CLAUDE.md 复制到项目根目录
cp ./doramagic-output/python-dotenv/CLAUDE.md /path/to/your/project/CLAUDE.md

# 之后在项目中使用 Claude Code, AI 自动加载知识
```

加载到 Cursor IDE:

```
# 将 .cursorrules 复制到项目根目录
cp ./doramagic-output/python-dotenv/.cursorrules /path/to/your/project/.cursorrules

# Cursor 会自动读取并应用规则
```

作为 AI 顾问使用:

```
# advisor-brief.md 可直接复制到任何 AI 对话中作为上下文
cat ./doramagic-output/python-dotenv/advisor-brief.md | pbcopy
# 粘贴到 ChatGPT / Claude 对话中
```

8.5 高级用法

切换 LLM 提供商:

```
# 查看当前可用提供商
doramagic providers

# 使用本地 Ollama（无需 API Key，完全离线）
doramagic full ./my-project --provider ollama --model llama3.2

# 使用 Gemini
doramagic full ./my-project --provider gemini --model gemini-1.5-pro
```

查看当前配置:

```
doramagic config
```

---

9. 用户使用流程——OpenClaw Skill 版本

9.1 前置条件

- OpenClaw 平台已安装并运行（`openclaw-gateway` + `openclaw-node`）
- 技能已注册到 OpenClaw 工作空间

9.2 技能 1: Soul Extractor（灵魂提取）

触发方式（在 OpenClaw 对话中）:

```
用户: 帮我提取 python-dotenv 项目的灵魂
用户: doramagic extract https://github.com/theskumar/python-dotenv
用户: 提取这个项目的知识: https://github.com/theskumar/python-dotenv
```

执行流程:

```
1. 用户提供项目 URL 或名称
          │
2. 系统自动下载仓库并 Repomix 打包
          │
3. Stage 1: 本质发现
   │  系统问 7 个核心问题，自动回答
   │  产出: 设计哲学 + 心智模型
          │
4. Stage 2: 概念与工作流提取
   │  产出: 概念卡片 + 工作流卡片
          │
5. Stage 3: 决策规则提取
   │  产出: 代码规则 + 社区踩坑
          │
6. Stage 3.5: 验证门控 ★
   │  自动运行验证脚本
   │  PASS → 继续 | BLOCKED → 自动修复重试
          │
7. Stage 4: 专家叙事合成
   │  产出: 1500 词专家级项目介绍
          │
8. Stage M: 模块地图
   │  产出: 模块分解 + 依赖关系
          │
9. Stage C: 社区智慧
   │  产出: 社区经验 + 最佳实践
          │
10. Stage F: 组装输出
    │  产出: CLAUDE.md + .cursorrules + advisor-brief.md
          │
11. 自动转换为 OpenClaw 知识包
    │  SOUL.md + AGENTS.md + README.md
          │
   ✅ 完成 — 知识注入 OpenClaw 工作空间
```

用户可在任意阶段查看中间产出:

```
用户: 看一下目前提取的概念卡片
系统: [展示 CC-001 ~ CC-006 卡片内容]

用户: 验证通过了吗?
系统: Stage 3.5 验证结果: PASS (Faithfulness: 100, Coverage: 95, ...)
```

9.3 技能 2: Skill Forge (技能锻造)

触发方式:

```
用户: 我想要一个帮我管理环境变量的 AI 技能
用户: forge skill 健身追踪
用户: 锻造一个库存管理的技能
```

执行流程:

```
Stage 1: 需求发现（苏格拉底式对话）
   │  系统: "你想用这个技能解决什么具体问题？"
   │  系统: "你目前是怎么处理的？"
   │  系统: "理想状态是什么样的？"
   │  → 提炼用户真实需求本质
          │
Stage 2: 开源发现
   │  系统自动搜索 3-5 个相关开源项目
   │  → 评估相关性和适用性
          │
Stage 3: 灵魂提取
   │  对每个相关项目执行 soul-extractor
   │  → 提取设计哲学和心智模型
          │
Stage 4: 社区智慧合成
   │  综合多个项目的社区经验
   │  → 领域知识和常见陷阱
          │
Stage 5: 技能锻造
   │  基于提取的知识生成定制技能包
   │  → SKILL.md + stages/ + scripts/
          │
   ✅ 完成 — 输出可安装的 AI 技能包
```

9.4 OpenClaw 集成架构

```
OpenClaw 工作空间
├── SOUL.md          ← 从 advisor-brief.md 转换
├── AGENTS.md        ← 从模块地图生成的代理定义
├── IDENTITY.md      ← 工作空间身份配置
├── MEMORY.md        ← 持久化记忆
├── skills/
│   └── doramagic/
│       ├── SKILL.md         ← 技能定义
│       ├── run-doramagic.sh ← 执行脚本
│       └── make-openclaw-pack.mjs ← OpenClaw 格式转换
└── runtime/
    └── jobs/
        └── 20260311T012527Z-extract-python-dotenv/
            ├── soul.md
            ├── judge-report.md
            ├── cards/
            ├── modules/
            └── openclaw-pack/
                ├── SOUL.md
                ├── AGENTS.md
                └── README.md
```

---

10. 输出物说明

10.1 输出文件清单

文件	用途	目标用户
`soul.md`	项目灵魂摘要——身份、模块、推荐卡片索引	所有用户
`cards/CC-*.md`	概念卡片 (5-10 张)	开发者
`cards/WC-*.md`	工作流卡片 (≥1 张)	开发者
`cards/DR-*.md`	决策规则卡片 (≥8 张)	所有用户
`cards/CT-*.md`	合约卡片	开发者
`cards/AC-*.md`	架构卡片 (≥1 张)	技术决策者
`modules/M-*/soul.md`	模块灵魂	开发者
`modules/M-*/api.md`	模块 API 文档	开发者
`expert_narrative.md`	专家叙事 (≤1500 词)	所有用户
`community-wisdom.md`	社区智慧摘要	所有用户
`module-map.md`	模块地图 (含 Mermaid 图)	技术决策者
`judge-report.md`	质量评判报告	内部验证
`CLAUDE.md`	Claude Code 上下文注入文件	开发者
`.cursorrules`	Cursor IDE 规则文件	开发者
`advisor-brief.md`	AI 顾问简报——非技术用户指南	非技术用户

10.2 CLAUDE.md 结构

```
# {项目名} — AI 顾问知识

## 项目身份
(从 00-soul.md 提取)

## 设计哲学
(从 expert_narrative.md 提取)

## 心智模型
(从 expert_narrative.md 提取)

## 关键决策规则
(从 DR-* 卡片汇总，仅 severity=critical 的规则)

## 你一定会踩的坑
(从 community-wisdom.md 提取前 5 个最高频痛点)

## 模块地图
(从 module-map.md 提取)

## 知识卡片索引
(所有卡片的 ID + 标题列表)
```

10.3 advisor-brief.md 结构

```
# {项目名} — 你的 AI 顾问知道这些

## 这个项目是什么?
(一段非技术语言的项目介绍)

## 它能帮你做什么?
(列出核心能力)

## 使用时要注意什么?
(社区踩坑经验，用故事形式呈现)

## 常见问题
(基于社区 Issue 整理的 FAQ)

## 不适合的场景
(项目的明确边界和限制)
```

---

11. 技术栈与依赖

11.1 核心技术栈

组件	技术	版本	说明
运行时	Node.js	≥ 18.0	ES Module
语言	TypeScript	5.x	严格模式
CLI 框架	Commander.js	—	命令解析
代码打包	Repomix	—	仓库压缩 + XML 输出
LLM: Claude	@anthropic-ai/sdk	—	主要提供商
LLM: Gemini	@google/generative-ai	—	备选提供商
LLM: Ollama	HTTP API	—	本地离线提供商
数据验证	Zod	—	Schema 校验
配置	js-yaml	—	YAML 解析
文件匹配	glob	—	文件发现
进度显示	ora + picocolors	—	CLI UX
ID 生成	uuid	—	唯一标识
测试	Vitest	—	单元测试框架
代码质量	ESLint	—	Lint

11.2 外部依赖

依赖	用途	必需?
Repomix	代码打包（compress mode, XML output）	✅ 是
GitHub CLI (`gh`)	社区数据获取 (Issues/PRs)	仅 community 命令需要
LLM API Key	Claude/Gemini 的 API 访问	取决于选择的提供商
Ollama	本地模型运行	仅 Ollama 提供商需要

11.3 脚本生态

脚本	功能	阶段
`prepare-repo.sh`	下载 → 解压 → Repomix 打包 → 事实提取	Stage 0
`extract.py`	全流程编排（含阶段检查点）	全局
`extract_repo_facts.py`	确定性事实提取（非 LLM）	Stage 0
`collect-community-signals.py`	GitHub/SO 信号收集	Stage C
`validate_extraction.py`	事实核查门控 + 可追溯性验证	Stage 3.5
`validate_output.py`	输出 Schema 和内容验证	Stage F
`assemble-output.sh`	合并阶段产出为最终交付物	Stage F

---

12. 验证数据与实验结论

12.1 核心实验: exp01 (python-dotenv)

目标项目: python-dotenv (1,105 行 Python, BSD-3)
执行时间: ~40 分钟 (全自动)
估计成本: ~$2-5

A/B/C 三组对照结果

指标	A 组 (无灵魂)	B 组 (代码半魂)	C 组 (完整灵魂)
平均得分 (满分25)	10.6	20.7	23.9
百分制均分	42%	83%	96%
改善幅度 (vs A)	—	+96%	+125%

5 维度详细评分

维度	A 组	B 组	C 组
接地性 (Groundedness)	1.8	4.4	5.0
具体性 (Specificity)	1.0	4.3	5.0
有用性 (Usefulness)	2.6	4.0	4.9
安全性 (Risk)	2.8	4.3	5.0
不确定性管理 (Uncertainty)	2.5	3.8	4.0

关键发现

1. **幻觉陷阱完美拦截**: Q7 (bare $VAR 插值) — 无灵魂 AI 自信给出错误答案 (6/25)，有灵魂后正确回答 (24/25)
2. **社区半魂在集成/部署场景最有价值**: Q9 (Docker 兼容) 从 12 分飙升到 24 分 (+12)
3. **代码半魂对纯代码问题已足够**: Q5/Q7/Q8 社区半魂增益为 0

产出统计

产出	数量
概念卡片 (CC)	3
工作流卡片 (WC)	3
代码规则 (DR-001~005)	5
社区规则 (DR-100~120)	21
项目灵魂 (project_soul.md)	1
专家叙事 (expert_narrative.md)	1
评判报告 (judge_report.md)	1

12.2 假说验证状态

假说	结果	证据
H1: 灵魂提取提升 AI 质量	**PASS** ✅	42% → 96%
H2: 代码半魂单独有效	**PASS** ✅	42% → 83%
H3: 双半魂互补非冗余	**PASS** ✅	A/B/C 三组各有贡献领域
H4: 幻觉可消除	**PASS** ✅	2 个幻觉陷阱问题完美拦截
H5: 过程成本可控	**PASS** ✅	~40 分钟, ~$2-5
H6: 中等项目有效	⏳ 待验证	需测试 5K-15K LOC 项目
H7: 大型项目可扩展	⏳ 待验证	大型 monorepo 未测试

12.3 多模型实现验收 (2026-03-11)

版本	开发者	状态	代码量
A: Skill 定义	Claude Code (本机)	✅ v1.0 完成	~700 md + ~2200 脚本
B: doramagic-claude	Claude Opus (251)	⚠️ 60% 完成	2,272 行 TS
C: doramagic-codex	GPT-5.3 Codex (251)	✅ **95% 完成**	4,621 行 TS
D: doramagic-python	Gemini (251)	🔴 仅框架	131 行
E: GLM 版本	GLM 5.0	🔴 不存在	—

合并决策: 以 doramagic-codex 为基线，集成本机 Skill 定义层，cherry-pick doramagic-claude 的 agent harness 模式。

12.4 doramagic-codex 测试结果

测试项目	判定	忠实度	覆盖度	可追溯性	一致性	可操作性
superpowers-final-v2	PASS	100	100	100	100	100
doramagic-self-full-v2	PASS	100	100	100	100	100
doramagic-self-full-v3	PASS	100	100	100	100	100
doramagic-self-snapshot	PASS	100	100	100	100	100

---

13. 已知限制与待解决问题

13.1 技术限制

限制	影响	缓解策略
大型仓库 token 限制	>50K LOC 项目可能超出 LLM 上下文	Repomix compress 模式 + 模块化提取
自评估偏差	同一模型提取又评判可能有偏	计划引入交叉模型评判
Stack Overflow 数据获取	搜索返回空结果	改用 Google Custom Search API
Windows 路径兼容性	部分脚本假设 Unix 路径	需适配
仅支持文本代码	二进制文件、图片、视频无法处理	v2 规划多模态扩展

13.2 未解决的核心问题

#	问题	状态	说明
1	"差不多正确"陷阱	🔴 未解决	非技术用户无法辨别微妙错误的 AI 回答
2	商业模型	🔴 未定义	定价、单位经济学、每项目成本
3	冷启动选品	🟡 部分解决	有选品标准但无优先级排名
4	doramagic-codex 独立性	🟡 待处理	目前嵌套在 WhisperX git 仓库中
5	Stage M/C 端到端验证	🟡 待测试	代码存在但未在复杂项目上验证
6	MiniMax 跳过 assemble-output.sh	🔴 已知	FEATURE INVENTORY 未生效

13.3 实验中发现的 6 个矛盾

#	矛盾	影响
1	战略说"社区更重要"，战术说"代码优先"	优先级冲突
2	DeepWiki: 立即使用 vs 可选 vs 依赖	架构冲突
3	"1 API Key + SQLite" vs 实际架构复杂度	演进路径不明
4	云端采集 vs 本地服务定位	边界模糊
5	Markdown vs JSON vs XML 格式选择	已决议: Markdown + YAML
6	YouTube 作为"金矿" vs 字幕获取不可行	可行性缺口

---

14. 产品路线图

14.1 v1.0 — 当前版本 (验证完成)

- [x] 核心提取管道 (Stage 0–F)
- [x] 五种知识卡片体系
- [x] 三个 LLM 提供商 (Claude/Gemini/Ollama)
- [x] 质量评判系统 (5 维评分)
- [x] exp01 验证通过 (42% → 96%)
- [x] CLI 6 命令完整
- [x] OpenClaw Skill 集成

14.2 v1.1 — 工程加固 (计划中)

- [ ] 将 doramagic-codex 从 WhisperX 仓库独立
- [ ] 集成本机 Skill 定义的 Stage M/C 到 CLI
- [ ] 中等项目验证 (pyjwt/itsdangerous, 5K–15K LOC)
- [ ] 交叉模型评判 (消除自评估偏差)
- [ ] "差不多正确"缓解策略实装
- [ ] 合并 doramagic-claude 的 agent harness 模式
- [ ] 清理冗余的 soul-extractor 副本

14.3 v2.0 — 平台化 (规划中)

- [ ] Web 界面 (灵魂注入 + 顾问查询)
- [ ] 大型 monorepo 支持 (分模块提取 + 合成)
- [ ] 知识图谱 (Neo4j) 用于跨项目知识关联
- [ ] 向量数据库 (Qdrant) 用于语义检索
- [ ] MCP Server 实装
- [ ] 多模态支持 (视频/图片/PDF 文档)
- [ ] DeepWiki 深度集成
- [ ] DSPy 自动 Prompt 优化
- [ ] 冷启动项目库 (Top 20 项目预提取)

14.4 v3.0 — 商业化 (远景)

- [ ] SaaS 平台
- [ ] API 服务
- [ ] 团队协作
- [ ] 企业私有部署
- [ ] 自动更新 (项目版本变更时重新提取)
- [ ] 技能市场 (skill-forge 生成的技能共享/交易)

---

15. 附录

附录 A: 项目文件目录

```
Doramagic/
├── docs/
│   ├── brainstorm/           # 6 轮头脑风暴文档
│   ├── research/             # 18 份研究报告
│   ├── plans/                # 3 份执行计划
│   ├── experiments/          # 实验框架模板
│   └── soul_document_template.md
│
├── experiments/              # 8 轮实验数据
│   ├── exp01/                # python-dotenv 基线验证 ✅
│   ├── exp01-v04-minimax/    # MiniMax 模型测试
│   ├── exp02-v05-minimax/    # MiniMax 迭代
│   ├── exp03~exp06/          # superpowers 项目迭代测试
│   ├── exp07/                # 用户测试
│   └── exp08/                # v0.9 + P0 补丁
│
├── skills/
│   ├── soul-extractor/       # 核心提取技能 v1.0
│   │   ├── SKILL.md
│   │   ├── stages/           # 7 个阶段定义
│   │   └── scripts/          # 7 个执行脚本
│   └── skill-forge/          # 技能锻造技能 v1.0
│       ├── SKILL.md
│       └── stages/           # 5 个阶段定义
│
├── scripts/                  # 原型脚本
├── reports/                  # 验收报告
├── research/                 # 外部研究论文和审查
└── test-run/                 # 测试环境

远程 251:
├── doramagic-codex/          # ★ 最完整 CLI (4,621 行 TS)
├── doramagic-claude/         # 60% 完成的 TS CLI
├── doramagic-python/         # 仅框架
└── .openclaw/
    ├── skills/doramagic/     # OpenClaw 注册技能
    └── workspace/doramagic/  # OpenClaw 工作空间
```

附录 B: 关键术语表

术语	定义
灵魂 (Soul)	项目的隐性知识总和——设计哲学、心智模型、决策规则、社区经验
双半魂模型	代码半魂 (内部逻辑) + 社区半魂 (实践经验) 的互补知识结构
Eagle Eye	第一阶段鸟瞰分析——识别项目身份、模块地图、灵魂焦点
Deep Dive	第二阶段深度提取——概念、工作流、决策规则
知识卡片	结构化知识单元，每张卡片有唯一 ID、证据溯源、YAML 前置
验证门控	Stage 3.5 的自动化质量检查点，不通过则阻断后续流程
Repomix	开源工具，将代码仓库压缩为单一 XML 文件供 LLM 消费
证据锚点	file:line 格式的代码引用，确保知识可追溯到源码
反理性化	防止 LLM 跳过不方便事实的检查机制
advisor-brief	面向非技术用户的 AI 顾问知识简报
Skill Forge	从用户需求出发，发现相关开源项目并锻造定制 AI 技能的元技能

附录 C: 配置参考

doramagic.yaml 完整参数:

```
llm:
  provider: claude | gemini | ollama  # LLM 提供商
  model: <model-name>                 # 模型名称
  base_url: <url>                     # Ollama 专用: 本地服务地址

extraction:
  max_concepts: 10                    # 最大概念卡片数 (默认 10)
  max_workflows: 5                    # 最大工作流卡片数 (默认 5)
  evidence_level: E1 | E2 | E3 | E4  # 最低证据级别 (默认 E2)

community:
  min_comments: 5                     # Issue 最少评论数 (默认 5)
  require_maintainer: true | false    # 是否要求维护者参与 (默认 true)
  max_threads: 100                    # 最大分析线程数 (默认 100)

output:
  format: markdown                    # 输出格式 (当前仅支持 markdown)
  language: zh-CN | en                # 输出语言 (默认 zh-CN)
```

环境变量覆盖:

```
ANTHROPIC_API_KEY      # Claude API Key
GOOGLE_AI_API_KEY      # Gemini API Key
DORAMAGIC_PROVIDER     # 覆盖 llm.provider
DORAMAGIC_MODEL        # 覆盖 llm.model
DORAMAGIC_OUTPUT       # 覆盖默认输出目录
```

附录 D: 实验评分矩阵 (exp01 完整数据)

问题	主题	A组	B组	C组	B增益	C增益
Q1	架构理解	11	23	24	+12	+1
Q2	变量插值	10	23	24	+13	+1
Q3	引号规则	12	22	24	+10	+2
Q4	加载流程	10	22	24	+12	+2
Q5	错误恢复	12	23	23	+11	0
Q6	Hash 陷阱	11	22	24	+11	+2
Q7	Bare $VAR 陷阱	**6**	**24**	**24**	**+18**	0
Q8	Override 双重效应	12	24	24	+12	0
Q9	Docker 兼容性	9	12	24	+3	**+12**
Q10	Debug 无效果	12	19	24	+7	+5
Q11	Flask+Docker	10	15	24	+5	**+9**
Q12	RSA+Win路径	12	19	24	+7	+5
均值	—	**10.6**	**20.7**	**23.9**	**+10.1**	**+3.3**

本手册基于 Doramagic 项目全部研究文档 (29份)、实验数据 (8轮)、代码实现 (3套) 和验收报告的综合整理。